

Building AI Agents

Turning LLMs into Decision Makers.

Discover  how LLMs evolve from prompt responders into systems capable of decision-making, tool execution, and coordination.

 Illustrated through practical implementations in .NET using Microsoft Agent Framework.

Agenda

01 The History that Led to Agentic Systems

Pag. 4

02 Technologies for Building Agentic Systems

Pag. 14

03 Demo

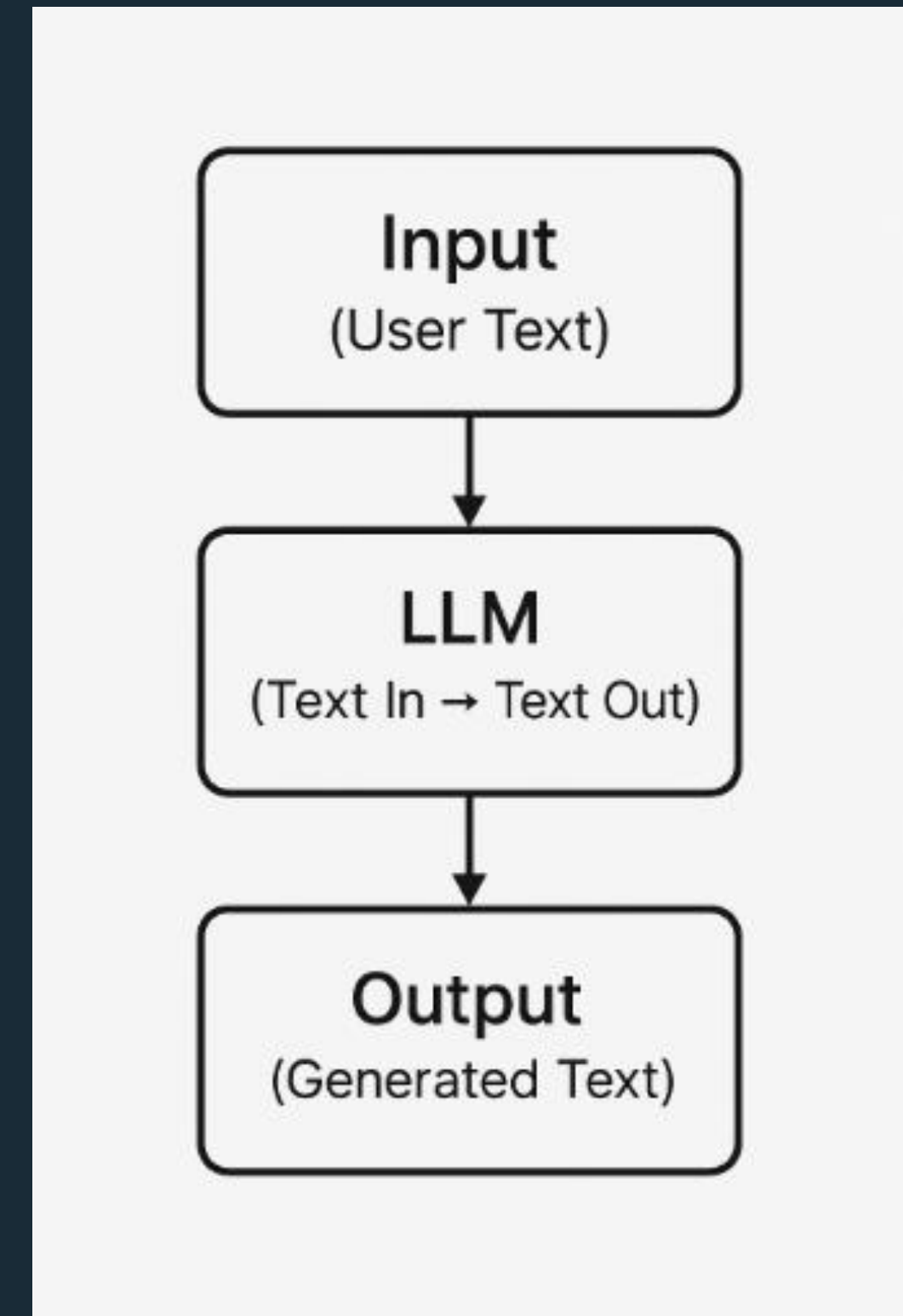
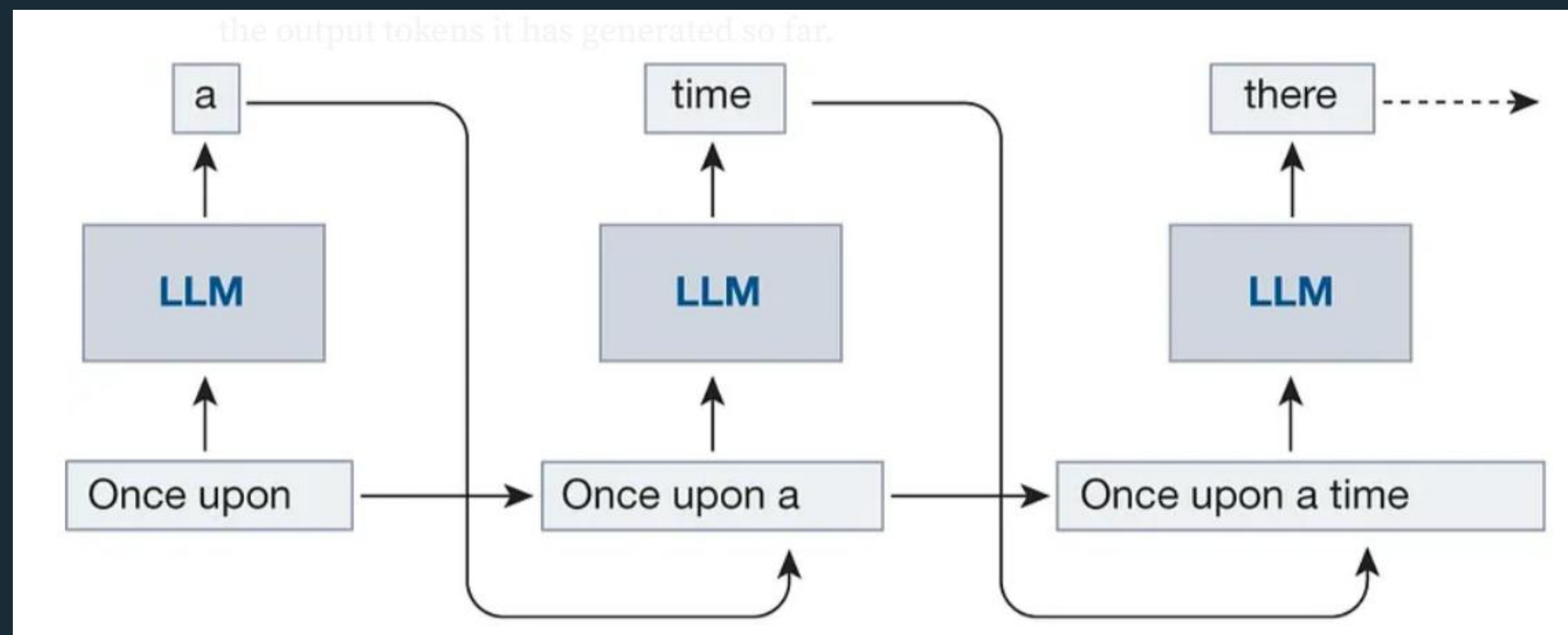
Pag. 19

01 The History that Led to Agentic Systems

Early LLM Capabilities

From Static Models to Interactive AI

1. LLMs are enormous neural networks (probabilistic model) that predict the next token/word
2. LLMs were originally "text-in, text-out"
3. Single-shot, stateless
4. No way to access live data
5. No awareness of external systems
6. No way to take actions



Fine-Tuning

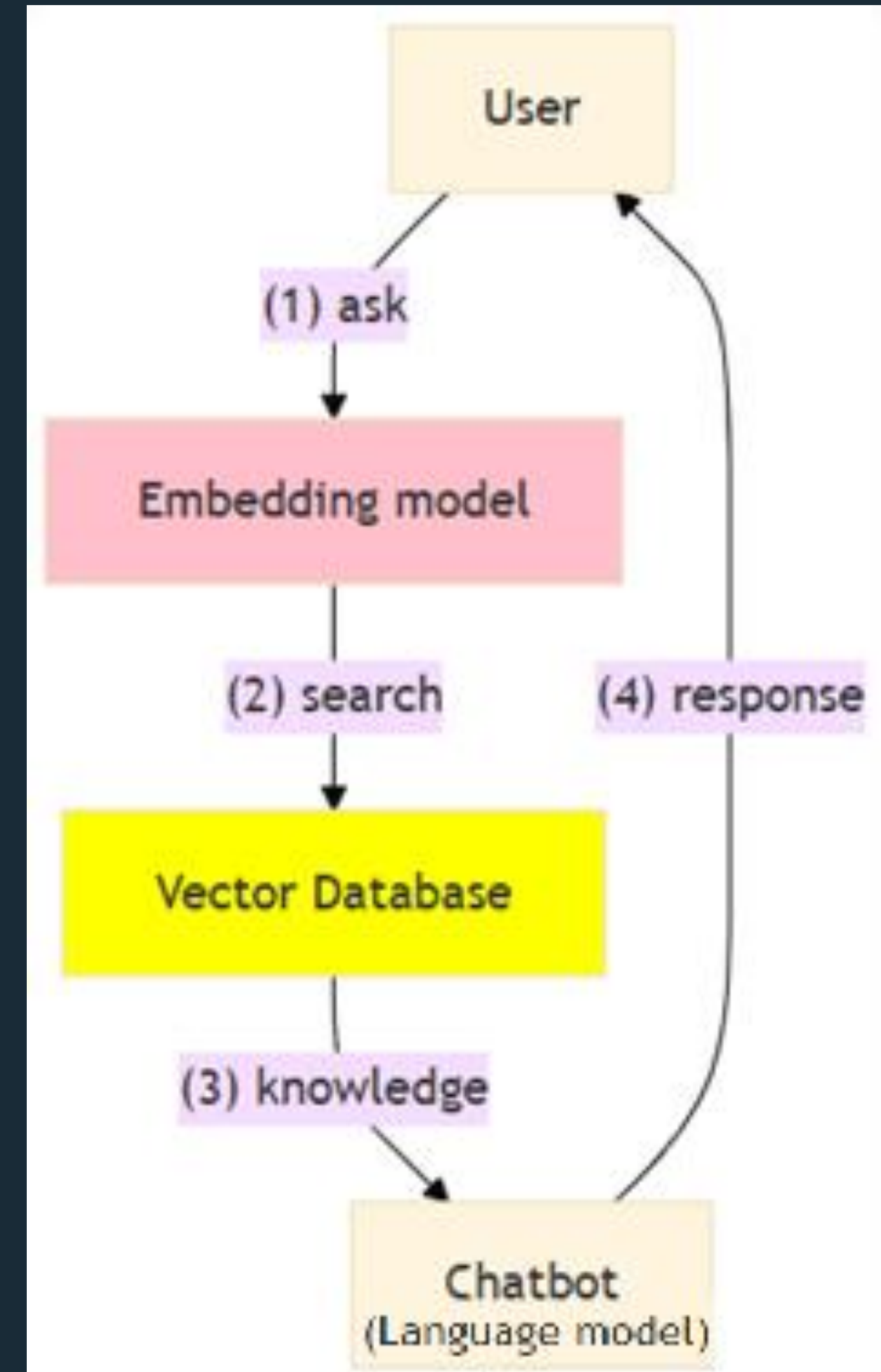
Trains the pre-trained LLM model with our specialized dataset: Helpful, but Limited

1. Customization via training
2. Expensive, slow iteration
3. Still no access to external systems
4. Still no way to take actions

RAG

Giving LLMs Knowledge – Retrieval-Augmented Generation

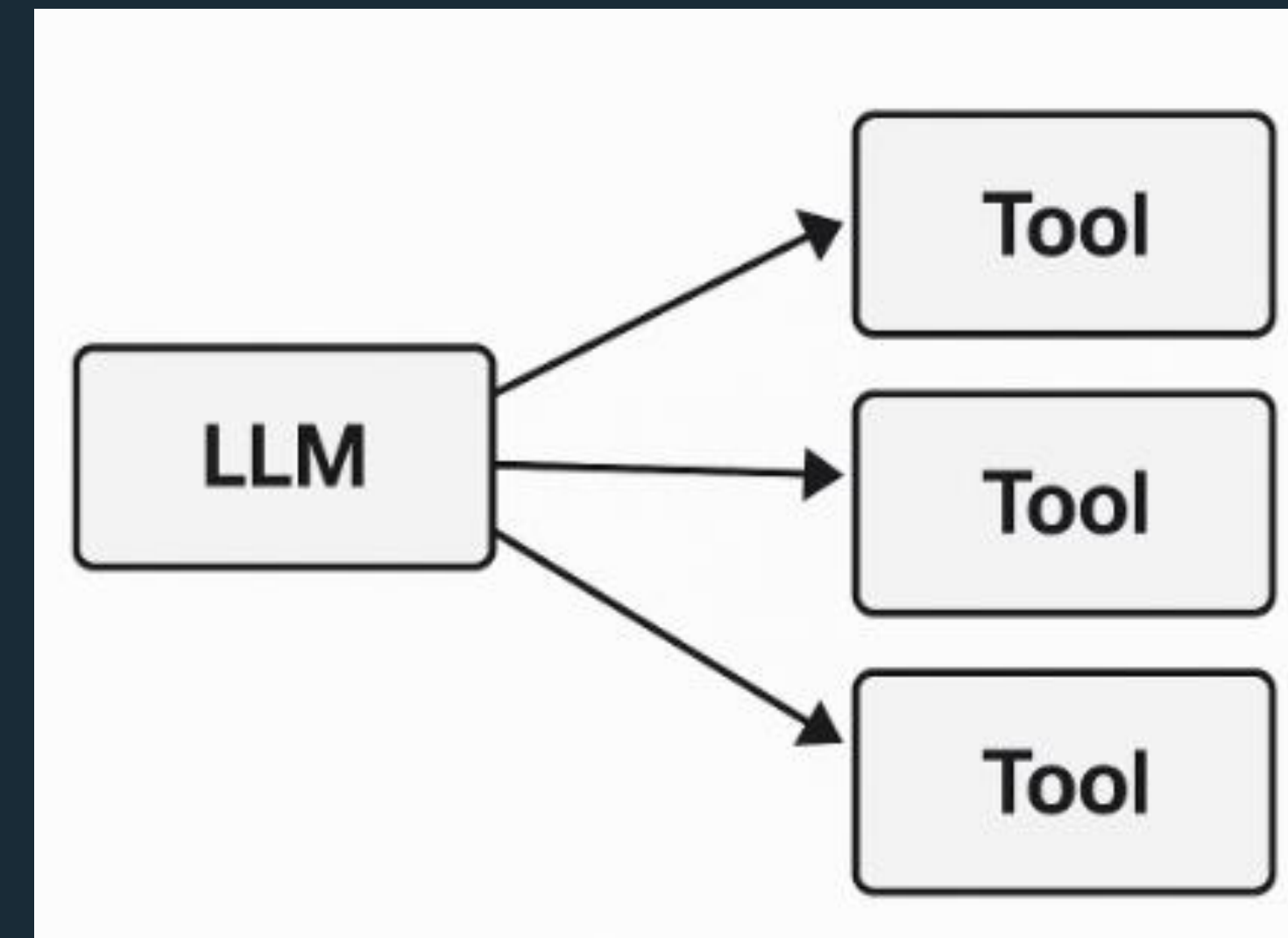
1. Adds knowledge beyond training
2. Still limited to passive retrieval
3. Read-only access
4. Still cannot act



Tool Use / Function Calling

LLMs Begin Taking Actions

1. LLMs call functions/tools via JSON schemas
2. Enabled real actions (APIs, DB calls, file ops)
3. But... every system defined tools differently
4. No standard, no permissions, no portability

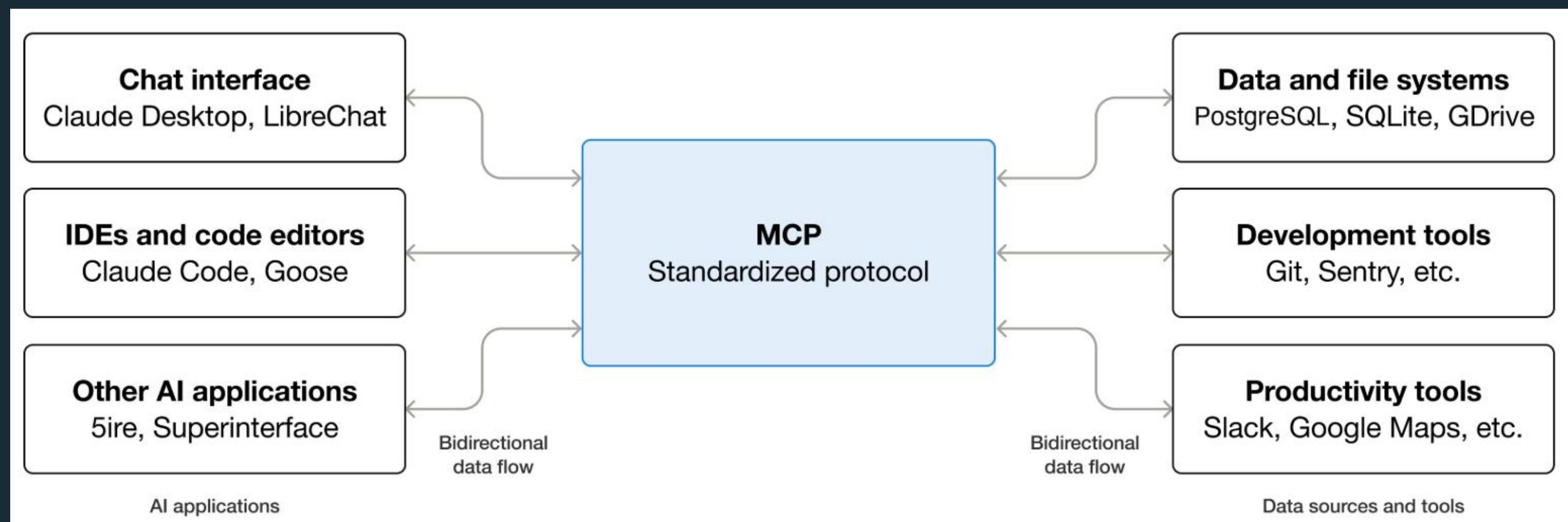


MCP

LLMs now has a standardized protocol for tools use / function calling

Model Context Protocol: Created by Anthropic <https://www.anthropic.com/news/model-context-protocol>

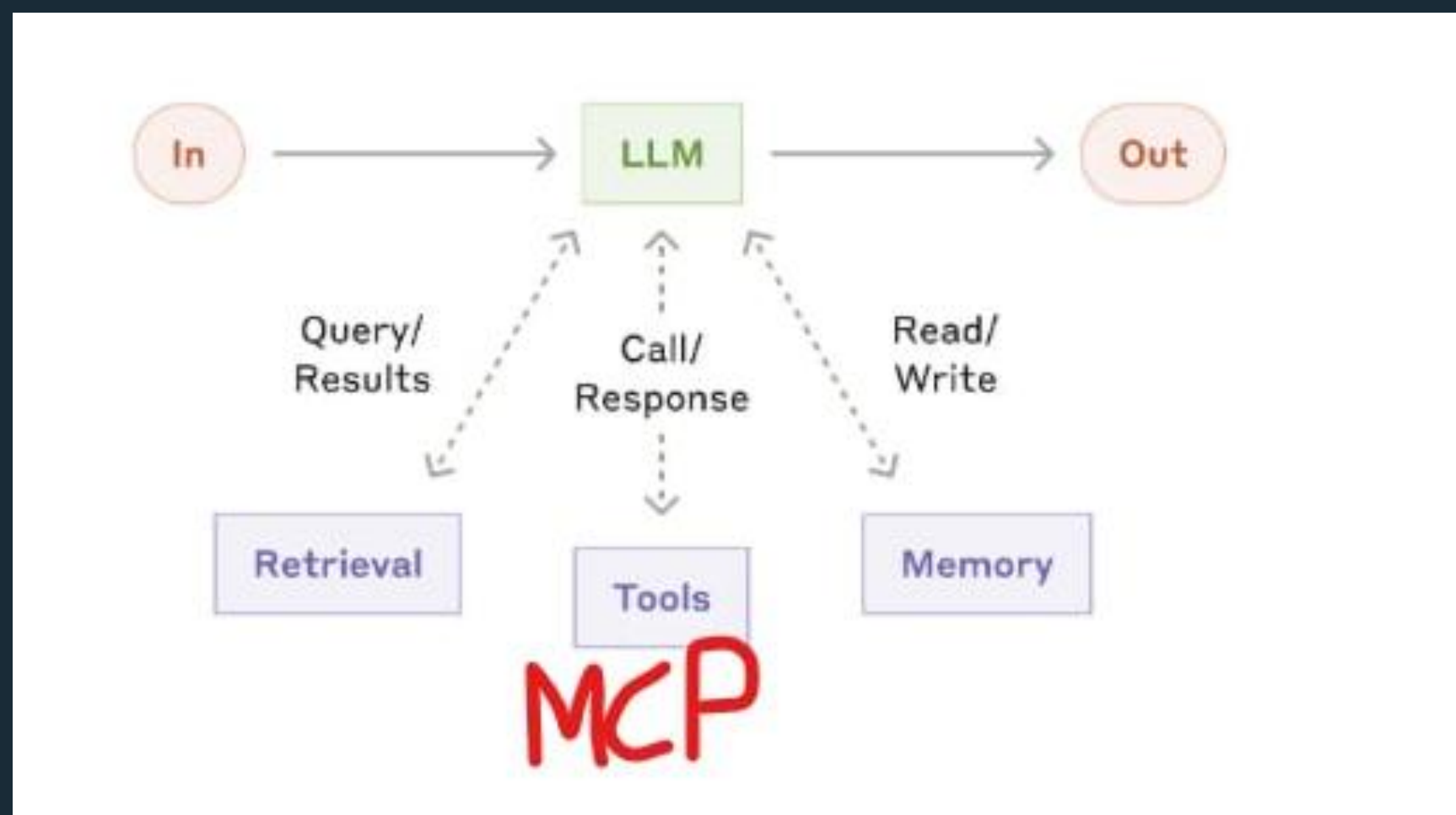
1. Uniform protocol for tools
2. Strong permission + security model
3. Works locally or remotely
4. Clear access control
5. Build once → use across platforms
6. Portable integrations (one server, many clients)



AI Agents

LLM is now a part of a bigger system. LLM is the brain of the AI Agent. Systems where large language models (LLMs) dynamically direct their own processes and tool usage, maintaining control over how they accomplish tasks

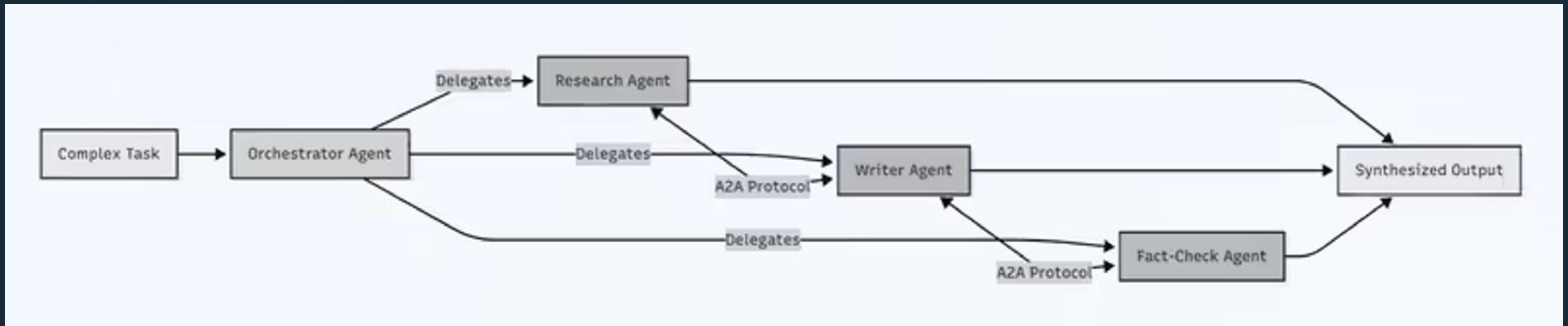
1. ReAct – reasoning and acting
2. Maintains state across steps (memory, session, scratchpad)
3. Selects tools, evaluates results, and adapts plans dynamically
4. Autonomy increases risk → requires constraints, observability, and limits



Multi-Agent Systems

From Single Decision-Maker to Coordinated Intelligence

1. Multiple agents collaborate, each with a specialized role or responsibility
2. Tasks are decomposed and delegated (planner, executor, verifier, etc.)
3. Agents coordinate via messaging, shared memory, or orchestration layers
4. Increased capability, but higher complexity: latency, cost, and failure modes multiply

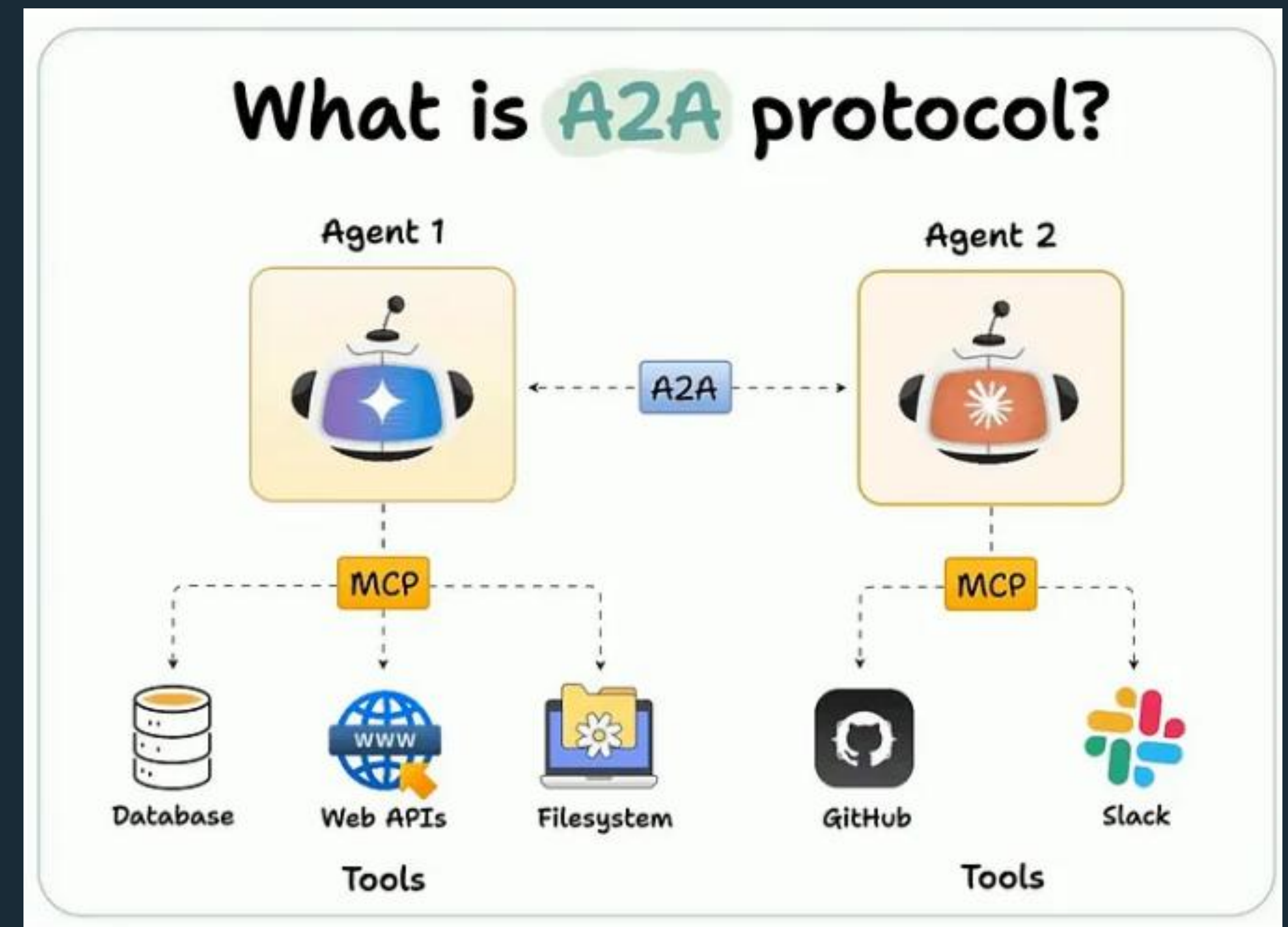


A2A

Standardized Communication Between Agents

Agent 2 Agent: Created by Google <http://a2a-protocol.org/>

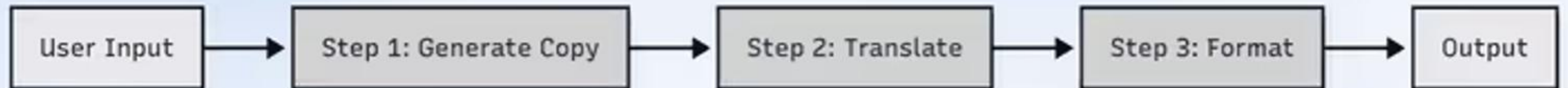
1. Enables agents to communicate across frameworks, runtimes, and deployments
2. Defines message formats, handshakes, and capability discovery
3. Supports interoperability in distributed, multi-vendor agent systems
4. Becomes critical as systems scale beyond single frameworks



Workflows

Deterministic Orchestration Over Autonomy

1. **Predefined steps with explicit control flow (sequential, branching, parallel)**
2. **Deterministic, testable, and easier to debug than agents**
3. **Ideal for stable business logic and well-known processes**

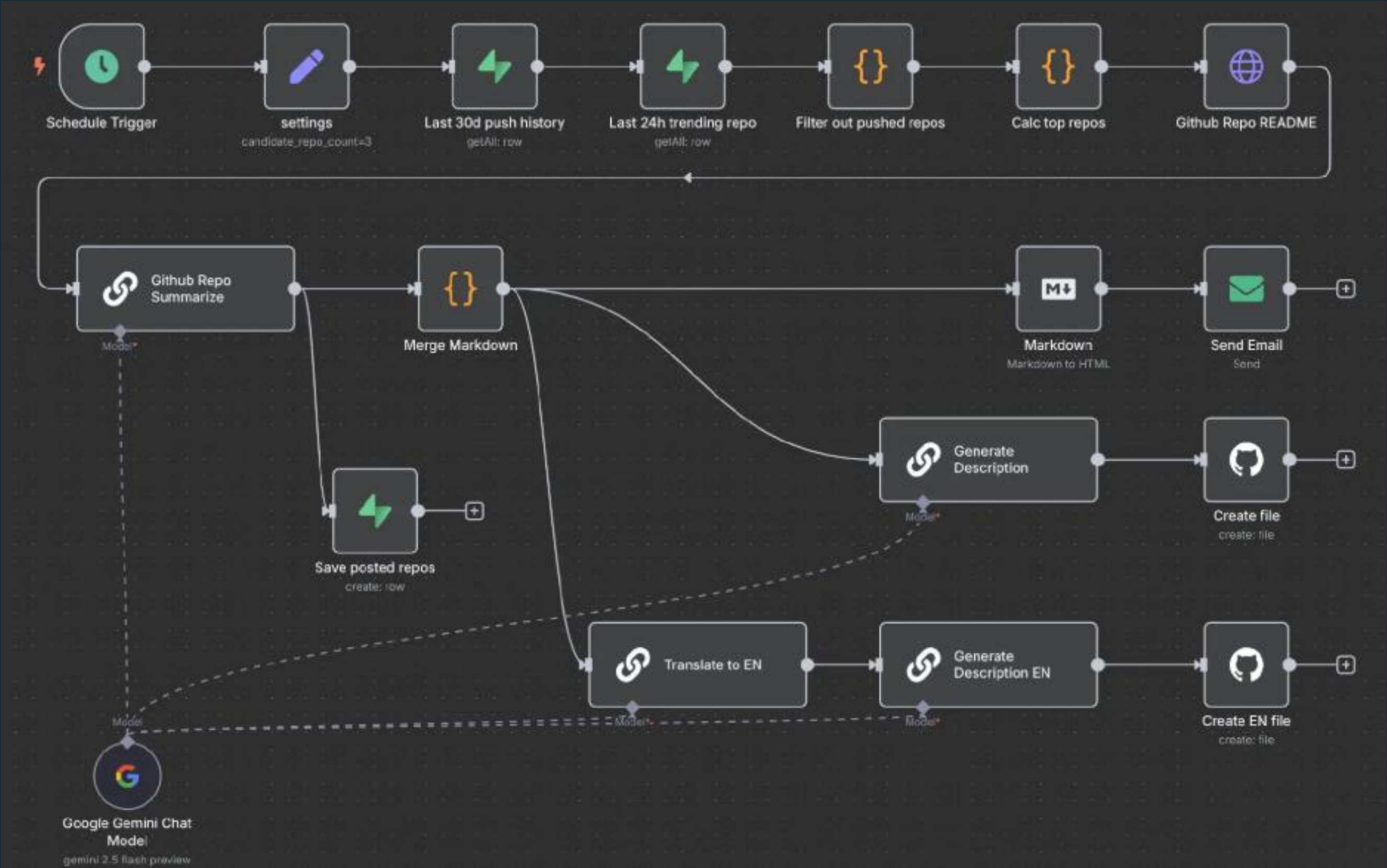


02 Technologies for Building Agentic Systems

No-code Agentic System Builders

No-code, or more precisely low-code, based on drag and drop.

- 1. Zapier
- 2. n8n
- 3. OpenAi Agent Builder



It's a Python's World

2025 was game changer. Everyone was building AI Agents.

In 2025 many of these were received official release status. And yes, AI development is Python's world, all the news are coming from the Python community.

1. LangChain
2. LangGraph
3. PydanticAI
4. CrewAI
5. Google Agent Development Kit (adk-python)
6. OpenAI Agents (openai-agents-python)
7. Anthropic Claude Agents (claude-agent-sdk-python)



LangChain



LangGraph

Microsoft AI Frameworks

Microsoft actively develops AI frameworks in both .NET and Python.

- 1. Semantic Kernel
- 2. AutoGen
- 3. Microsoft Agent Framework

	Semantic Kernel	AutoGen	Microsoft Agent Framework
Focus	Stable SDK with enterprise connectors, workflows, and observability	Experimental multi-agent orchestration from research	Unified SDK combining innovation + enterprise readiness
Interop	Plugins, connectors, and support for MCP, A2A, OpenAPI	Tool integration supported; lacks standardized cross-runtime protocols	Built-in connectors, MCP + A2A + OpenAPI
Memory	Multiple vector store connectors and memory store abstraction (e.g. Azure SQL Elasticsearch, MongoDB)	Support for in-memory / buffer history + external vector store memory options (ChromaDB, Mem0, etc)	Pluggable memory across stores (first-party and third-party), persistent & adaptive memory stored with retrieval, hybrid approaches
Orchestration	Deterministic + dynamic orchestration (Agent Framework, Process Framework)	Dynamic LLM orchestration (debate, reflection, facilitator/worker, group chat)	Deterministic + dynamic orchestration (Agent Orchestration, Workflow Orchestration)
Enterprise readiness	Telemetry, observability, compliance hooks	Minimal	Observability, approvals, CI/CD, long-running durability, hydration



Microsoft Agent Framework

The best from both worlds: Semantic Kernel + AutoGen.
MAF offers two primary categories of capabilities:

1. AI agents
2. Workflows



Chapter

03 Demo

Deploy LLM in Azure AI

My Visual Studio Benefits: <https://my.visualstudio.com/Benefits>

Azure AI (Microsoft Foundry): <https://ai.azure.com/>

AI Azure catalog: <https://ai.azure.com/catalog>

Models: <https://ai.azure.com/catalog/models>

Providers: <https://ai.azure.com/catalog/publishers>

Simple AI Agent

- Model
- Instructions
- Execute AI Agent
- Session
- Structured output

AI Agent using RAG

Just a simple mock RAG.

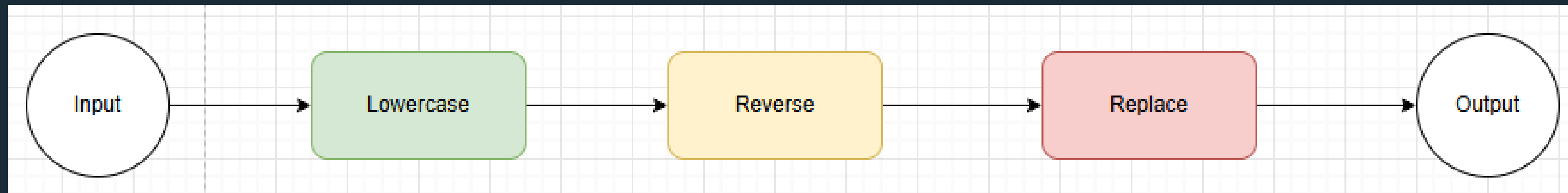
AI Agent using Tools

Functions as Tools

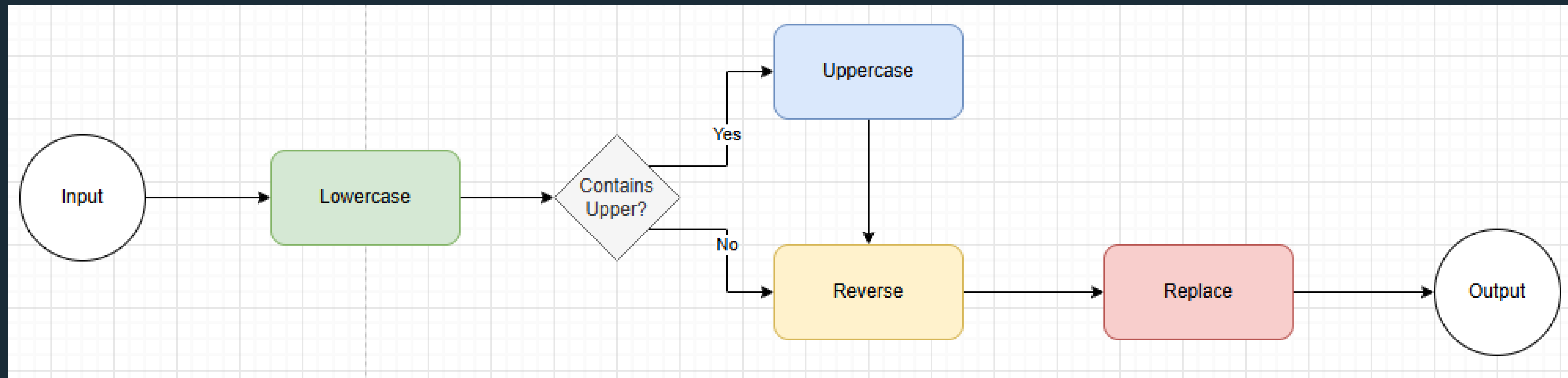
MCP as Tools

Workflows

Sequential Workflow

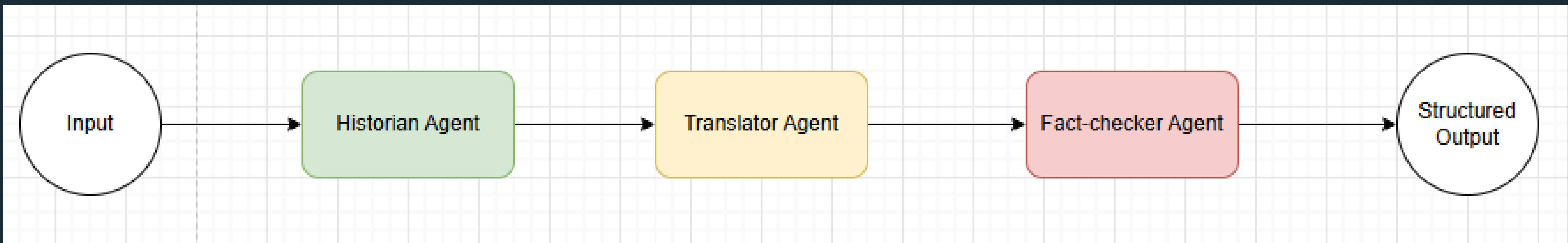


Branching Workflow



AI Agent Workflow

Workflow from 3 agents: Historian, Translator, Fact-Checker.



Final Notes

- 1. Don't build AI Agents for everything!!!
- 2. Keep it simple
- 3. Think like your agents

Checklist: “Should I build an agent”	
Is the task complex enough?	No → Workflows Yes → Agents
Is the task valuable enough?	<\$0.1 → Workflows >\$1 → Agents
Are all parts of the task doable?	No → Reduce scope Yes → Agents
What is the cost of error/error discovery?	High → Read-only/human-in-the-loop Low → Agents

Resources

Official Microsoft Agent Framework documentation:

<https://learn.microsoft.com/en-us/agent-framework/>

Thank *you*!